UNIVERSIDADE DE VILA VELHA

PROJETO E ANÁLISE DE ALGORITMOS

Explorando a intratabilidade e a força bruta no Problema do Caixeiro Viajante

Bernardo Ziviani Gasparino
Samuel Bruno da Silva
Yuri Luiz Kruger Torquato

Professor: Wanderson Muniz de Santana

Turma: CC7N

Vila Velha - ES
2026

Sumário

1 Introdução **2**

2 Fundamentação teórica **2**
2.1 Modelagem e força bruta 2
2.2 Fixação da origem e redução do espaço de busca 2

3 Implementação **2**
3.1 Algoritmo sequencial 2
3.2 Implementação paralela em GPU 3

4 Metodologia experimental **3**
4.1 Ambiente e configuração 3
4.2 Comandos utilizados 4

5 Resultados e validação **5**
5.1 Tabela de crescimento combinatório 5
5.2 Curva teórica e dados empíricos 6
5.3 Evidências de execução 8

6 Discussão das questões investigativas **8**
6.1 Redução do espaço de busca 8
6.2 Custo de avaliação por rota 9
6.3 Limiar prático da intratabilidade 9
6.4 Classes P e NP e heurísticas 10

7 Conclusão **10**

A Tempos individuais e dispersão **11**
A.1 Medições individuais: CPU, origem fixa 11
A.2 Medições individuais: GPU, origem fixa 12

1 Introdução

O Problema do Caixeiro Viajante, conhecido pela sigla TSP, consiste em encontrar um circuito de custo mínimo que visite cada cidade uma única vez e retorne à cidade de origem. Embora sua definição seja simples, a quantidade de rotas candidatas cresce rapidamente com o número de cidades. Este trabalho investiga esse crescimento por meio de uma implementação de força bruta, relacionando as medições experimentais à análise apresentada no material da disciplina [1] e aos objetivos da proposta de atividade [2].

O objetivo é implementar a busca exaustiva, registrar as quantidades de rotas e avaliações de arestas, medir o tempo para diferentes tamanhos de entrada e comparar os resultados com uma curva proporcional a $n!$.

2 Fundamentação teórica

2.1 Modelagem e força bruta

O conjunto de cidades é representado por $V = \{v_1, v_2, \ldots, v_n\}$, e o custo de deslocamento entre duas cidades é dado por $d(v_i, v_j)$. Na implementação, os custos são armazenados em uma matriz. Para cada rota candidata, o programa soma os custos das arestas percorridas e mantém a rota de menor custo encontrada.

Para evitar contar diferentes rotações do mesmo circuito, fixa-se v_1 como origem na execução principal. Restam, portanto, $n-1$ cidades a serem permutadas, resultando em

$$R(n) = (n-1)! \tag{1}$$

rotas. Cada rota possui n arestas, incluindo o retorno à origem. O número total de avaliações dessas arestas é

$$A(n) = n \cdot (n-1)! = n!. \tag{2}$$

Assim, para a implementação sequencial com avaliação linear por rota, o tempo de execução é $\Theta(n!)$ [1]. A igualdade acima representa o número de avaliações de arestas, não a quantidade total de instruções executadas pelo programa.

Como a matriz gerada neste projeto é simétrica, uma rota e sua inversa têm o mesmo custo. Mesmo assim, ambas são avaliadas separadamente, preservando a contagem $(n-1)!$ utilizada no material. Para $n = 6$, por exemplo, são $5! = 120$ rotas e $6 \cdot 120 = 720$ avaliações de arestas.

2.2 Fixação da origem e redução do espaço de busca

Sem fixar a origem, a enumeração de todas as permutações das n cidades produziria $n!$ representações. Ao fixá-la, o total é reduzido para $(n-1)!$. Como $n! = n \cdot (n-1)!$, a redução é por um fator igual a n. As diferentes escolhas de origem correspondem a rotações de um mesmo circuito e não alteram seu custo ótimo.

3 Implementação

3.1 Algoritmo sequencial

A implementação em Go gera uma matriz simétrica de custos positivos a partir de uma semente e enumera recursivamente as permutações das cidades restantes. Para cada

permutação, constrói o circuito, avalia suas n arestas e atualiza a melhor solução quando encontra um custo inferior. O programa registra a rota ótima, o custo, as contagens e o tempo de busca em arquivos CSV e de log com nomes únicos. O código-fonte está disponível no repositório do projeto [3].

3.2 Implementação paralela em GPU

Além da versão sequencial, o projeto possui uma implementação em CUDA, integrada ao programa em Go por meio de CGO. A matriz de custos é preparada no hospedeiro e copiada para a memória da GPU. A enumeração das $(n-1)!$ permutações é distribuída entre as threads CUDA: cada thread calcula custos de rotas e participa da redução do menor custo. Como uma redução paralela identifica o custo mínimo, mas não necessariamente a rota que o produziu, uma segunda passagem procura o índice da primeira rota com esse custo. A partir desse índice, o programa reconstrói a rota ótima e devolve o resultado ao Go.

O tempo da versão GPU registrado pelo programa inclui a chamada à rotina CUDA, alocação e transferência de dados, as duas passagens de busca e a sincronização necessária para obter os resultados no hospedeiro. Portanto, especialmente para entradas pequenas, o custo fixo de inicialização e transferência pode ser maior que o tempo efetivamente gasto calculando rotas. A versão é compilada com a tag `gpu`; por exemplo, `make gurun ARGS=-min-n 3 -max-n 16"`. A compilação requer CUDA/NVIDIA e as ferramentas configuradas no `Makefile`.

4 Metodologia experimental

4.1 Ambiente e configuração

Os experimentos foram executados em computadores distintos para CPU e GPU. As configurações informadas são:

- CPU (notebook): AMD Ryzen 7 4800H, 20 GB de RAM e EndeavourOS (baseado em Arch Linux);
- GPU: AMD Ryzen 7 5700G, 32 GB de RAM e NVIDIA GeForce RTX 5060 Ti com 16 GB, usando CachyOS;
- Go 1.27.3 nas execuções; CUDA 13.4.92 na execução GPU;
- CPU: $n = 3$ a $n = 15$; GPU: $n = 3$ a $n = 17$; origem fixa na cidade 1 e semente 42.

O tempo registrado compreende a enumeração, a montagem e a avaliação das rotas, incluindo a instrumentação. Na GPU inclui as duas passagens. A geração da matriz e a escrita do CSV não fazem parte desse intervalo. O registro detalhado de todas as rotas não deve ser ativado nos testes de desempenho, pois a escrita no log pode afetar os tempos.

Para cada tamanho n, o programa enumera as rotas e registra o tempo de busca. Quando existem várias execuções para o mesmo tamanho e dispositivo, os tempos são resumidos pela mediana: ordenam-se as amostras e toma-se o valor central (ou a média dos dois centrais, se a quantidade for par). Foram consideradas seis execuções de CPU para $n = 3$ a $n = 10$, entre quatro e cinco para $n = 11$ a $n = 14$, e uma para $n = 15$. Na

GPU, há duas amostras para $n = 3$ a $n = 16$ e uma para $n = 17$. Portanto, os resultados para $n = 15$ na CPU e $n = 17$ na GPU são medições únicas, não medianas robustas. Os comandos para reproduzir a agregação estão na Seção 4.2; as tabelas e os gráficos deste relatório já apresentam os valores calculados, sem exigir acesso aos arquivos CSV.

4.2 Comandos utilizados

Para a execução principal com origem fixa, por exemplo:

```
make run ARGS="-min-n 3 -max-n 10"
```

Para gerar o gráfico e a tabela a partir do CSV obtido:

```
make grafico RESULTADO="nome_do_arquivo.csv"
```

O CSV é lido em `resultados/`, e o gráfico PNG e a tabela são gerados em `grafricos/`. Os nomes dos arquivos são únicos.

Para medir empiricamente todas as origens, inclusive suas rotações redundantes, executa-se, por exemplo:

```
make run-random ARGS="-min-n 3 -max-n 15"
make gurun-random ARGS="-min-n 3 -max-n 17"
```

Apesar do nome histórico dos comandos, eles usam `-all-starts`: para cada n, executam a busca a partir de cada cidade, enumerando $n!$ representações distintas. Isso difere de `-random-start`, que sorteia uma única origem e ainda gera apenas $(n-1)!$ rotas. Os resultados sem fixação ficam em arquivos `sem_fixar_cpu_*.csv` e `sem_fixar_gpu_*.csv`, separados dos resultados com origem fixa. O tempo registrado é a soma dos tempos das buscas por origem; na GPU, cada busca inclui duas passagens e seus custos de CUDA. Após novas medições, regenere o apêndice e compile o relatório novamente. Os testes de maior tamanho podem levar muitas horas ou dias.

Para agregar todas as execuções disponíveis e produzir gráficos separados com mediana:

```
make grafico-mediana DISPOSITIVO=cpu
make grafico-mediana DISPOSITIVO=gpu
```

O comando lê `resultados/resultados*.csv` para CPU e `resultados/results_gpu*.csv` para GPU. Gera `Grafico_CPU_mediana.png` ou `Grafico_GPU_mediana.png` e a tabela CSV correspondente em `grafricos/`. Cada tabela inclui o número de amostras usadas por tamanho.

O Apêndice A apresenta todos os tempos individuais, seus arquivos de origem e os valores mínimo, mediano e máximo por tamanho. Ele pode ser regenerado com `python3 tabela_amostras.py`. A agregação reúne todos os CSVs encontrados por dispositivo; não é possível confirmar pelos CSVs isoladamente se todas as versões do programa e condições de execução eram idênticas. Os intervalos mostram a variação observada, não um intervalo de confiança. Além disso, a matriz é gerada com dimensão `max-n` antes de extrair as submatrizes: a mesma semente com valores diferentes de `max-n` não garante instâncias iguais. Para reprodução e comparação controlada, devem-se registrar o comando completo e a versão do código, usar a mesma matriz e manter as condições de execução constantes.

Para gerar a comparação teórica entre fixar e não fixar a origem, sem executar outro benchmark:

```
make grafico-reducao
```

Esse comando calcula os fatoriais e gera o gráfico e a tabela CSV em `grafricos/`.

5 Resultados e validação

5.1 Tabela de crescimento combinatório

A Tabela 1 e a Tabela 2 apresentam as contagens teóricas por passagem e os tempos resumidos por mediana, em milissegundos. A comparação com os registros confirmou, em todas as entradas, $(n-1)!$ rotas e $n!$ avaliações de arestas na CPU e na primeira passagem da GPU. Por exemplo, para $n = 10$ foram registradas 362.880 rotas e 3.628.800 avaliações de arestas. Os CSVs históricos da GPU contabilizam apenas a primeira passagem, embora o tempo inclua duas. A segunda passagem reavalia todas as rotas; portanto, o total efetivo das duas passagens é $2(n-1)!$ avaliações de rotas e $2n!$ avaliações de arestas. Os tempos resumem as amostras disponíveis; os valores individuais constam no apêndice.

Tabela 1: Resultados da execução em CPU: contagens teóricas e mediana do tempo.

n	$(n-1)!$	$n!$	Amostras	Mediana (ms)
3	2	6	6	0,000922
4	6	24	6	0,000547
5	24	120	6	0,001182
6	120	720	6	0,005270
7	720	5.040	6	0,031848
8	5.040	40.320	6	0,323821
9	40.320	362.880	6	3,124497
10	362.880	3.628.800	6	22,118782
11	3.628.800	39.916.800	5	234,589688
12	39.916.800	479.001.600	5	2.675,783333
13	479.001.600	6.227.020.800	4	31.836,321137
14	6.227.020.800	87.178.291.200	4	433.055,072678
15	87.178.291.200	1.307.674.368.000	1	5.981.114,958103

Tabela 2: Resultados da execução em GPU: contagens teóricas e mediana do tempo.

n	$(n-1)!$	$n!$	Amostras	Mediana (ms)
3	2	6	2	151,951220
4	6	24	2	0,211461
5	24	120	2	0,737821
6	120	720	2	0,348593
7	720	5.040	2	0,184059
8	5.040	40.320	2	0,283575
9	40.320	362.880	2	0,229065
10	362.880	3.628.800	2	0,571896
11	3.628.800	39.916.800	2	3,330259
12	39.916.800	479.001.600	2	32,461620
13	479.001.600	6.227.020.800	2	402,219234
14	6.227.020.800	87.178.291.200	2	6.073,329214

n	$(n-1)!$	$n!$	Amostras	Mediana (ms)
15	87.178.291.200	1.307.674.368.000	2	104.752,111644
16	1.307.674.368.000	20.922.789.888.000	2	1.592.764,836502
17	20.922.789.888.000	355.687.428.096.000	1	27.074.814,512025

5.2 Curva teórica e dados empíricos

Cada gráfico sobrepõe o tempo medido à função $f(n) = c \cdot n!$. A constante c é ajustada pelo método dos mínimos quadrados aos tempos em milissegundos, considerando uma reta que passa pela origem. Nos gráficos agregados, t_i é a mediana das amostras disponíveis para o tamanho n_i; quando existe apenas uma amostra, a mediana coincide com esse único valor. Para os pares $(x_i, y_i) = (n_i!, t_i)$, o valor utilizado é

$$c = \frac{\sum_i x_i y_i}{\sum_i x_i^2}. \tag{3}$$

O ajuste é realizado sobre os tempos em escala linear; consequentemente, as maiores medições têm maior influência. O gráfico utiliza escala logarítmica no eixo vertical apenas para facilitar a visualização de valores de ordens de grandeza diferentes. A proximidade entre as curvas ajuda na comparação, mas não constitui, isoladamente, uma prova da complexidade assintótica.

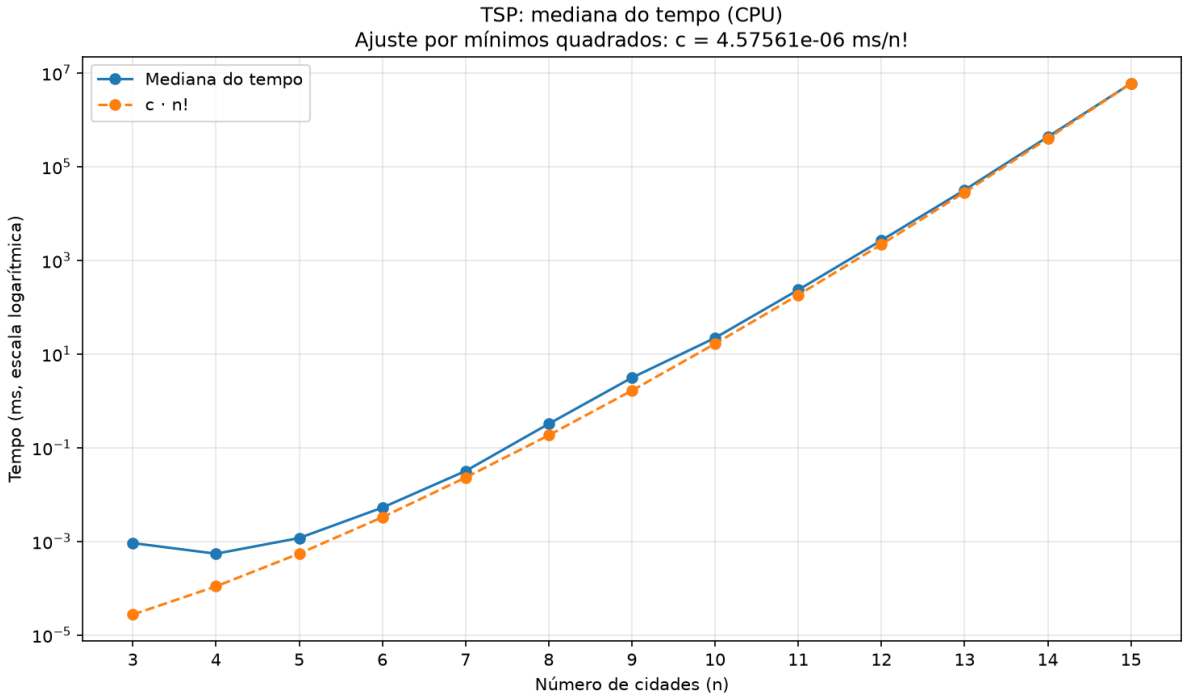


Figura 1: Mediana dos tempos da CPU e ajuste fatorial.

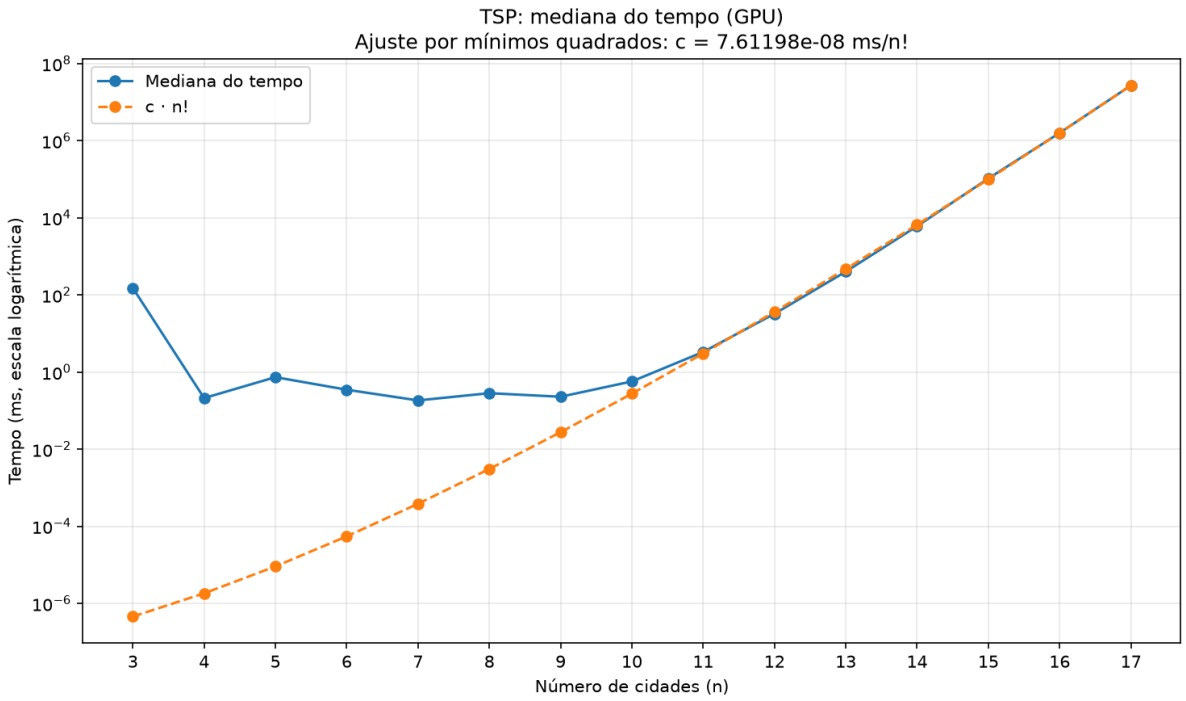


Figura 2: Mediana dos tempos da GPU e ajuste fatorial.

Na execução em CPU (Figura 1), o gráfico abrange $n = 3$ a $n = 15$ e o ajuste às medianas informa $c = 4{,}57561 \times 10^{-6}$ ms/$n!$. Na execução em GPU (Figura 2), o intervalo é $n = 3$ a $n = 17$ e o ajuste informa $c = 7{,}61198 \times 10^{-8}$ ms/$n!$. Esses coeficientes são ajustes independentes, calculados sobre os dados de cada execução, e não constituem uma razão de desempenho entre CPU e GPU.

Como exemplos concretos da tabela, em $n = 10$ a mediana foi de 22,118782 ms na CPU (seis amostras) e 0,571896 ms na GPU (duas amostras). Na CPU, $n = 14$ levou em mediana 433.055,072678 ms, aproximadamente 7 min 13 s, considerando quatro amostras. O maior caso concluído na GPU, $n = 17$, levou 27.074.814,512025 ms, aproximadamente 7 h 31 min, mas esse valor veio de uma única execução. A mediana de uma só amostra é apenas o tempo observado naquela execução.

As medições foram realizadas em computadores diferentes, portanto os gráficos não isolam o efeito do dispositivo: processadores, memória e sistemas operacionais também diferem. O tempo para $n = 3$ na GPU é alto em relação aos tamanhos seguintes, compatível com o custo fixo de inicialização e transferência CUDA. Para comparação controlada de desempenho, seria necessário executar ambas as versões na mesma máquina, com as mesmas instâncias e várias repetições. O ajuste fatorial descreve o crescimento; não demonstra, por si só, que os tempos de CPU e GPU sejam comparáveis.

5.3 Evidências de execução

```
2026/11/02 12:22:08 TSP por força bruta | origem aleatória: false | semente dos custos: 42
2026/11/02 12:22:08 Resultados serão gravados em resultados
2026/11/02 12:22:08 n=3 | rotas=2 (teórico=2) | arestas=6 | custo mínimo=147 | rota=1 → 2 → 3 → 1 | tempo=701ns
2026/11/02 12:22:08 n=4 | rotas=6 (teórico=6) | arestas=24 | custo mínimo=148 | rota=1 → 2 → 3 → 4 → 1 | tempo=471ns
2026/11/02 12:22:08 n=5 | rotas=24 (teórico=24) | arestas=120 | custo mínimo=138 | rota=1 → 2 → 3 → 4 → 5 → 1 | tempo=1.052µs
2026/11/02 12:22:08 n=6 | rotas=120 (teórico=120) | arestas=720 | custo mínimo=123 | rota=1 → 2 → 6 → 4 → 3 → 5 → 1 | tempo=4.709µs
2026/11/02 12:22:08 n=7 | rotas=720 (teórico=720) | arestas=5040 | custo mínimo=162 | rota=1 → 2 → 3 → 5 → 4 → 6 → 7 → 1 | tempo=26.098µs
2026/11/02 12:22:08 n=8 | rotas=5040 (teórico=5040) | arestas=40320 | custo mínimo=117 | rota=1 → 2 → 8 → 3 → 5 → 4 → 6 → 7 → 1 | tempo=422.93µs
2026/11/02 12:22:08 n=9 | rotas=40320 (teórico=40320) | arestas=362880 | custo mínimo=128 | rota=1 → 2 → 8 → 5 → 4 → 3 → 9 → 6 → 7 → 1 | tempo=2.469023ms
2026/11/02 12:22:08 n=10 | rotas=362880 (teórico=362880) | arestas=3628800 | custo mínimo=146 | rota=1 → 2 → 7 → 6 → 9 → 3 → 4 → 5 → 8 → 10 → 1 |
tempo=20.703818ms
2026/11/02 12:22:08 n=11 | rotas=3628800 (teórico=3628800) | arestas=39916800 | custo mínimo=165 | rota=1 → 2 → 7 → 6 → 9 → 3 → 10 → 8 → 5 → 4 → 11 → 1 |
tempo=220.352922ms
2026/11/02 12:22:10 n=12 | rotas=39916800 (teórico=39916800) | arestas=479001600 | custo mínimo=191 | rota=1 → 2 → 7 → 6 → 11 → 4 → 5 → 8 → 10 → 3 → 9 → 12 →
1 | tempo=2.467607679s
2026/11/02 12:22:41 n=13 | rotas=479001600 (teórico=479001600) | arestas=6227020800 | custo mínimo=171 | rota=1 → 2 → 8 → 10 → 3 → 9 → 12 → 13 → 5 → 4 → 11 → 6
→ 7 → 1 | tempo=30.59922779s
2026/11/02 12:29:41 n=14 | rotas=6227020800 (teórico=6227020800) | arestas=87178291200 | custo mínimo=146 | rota=1 → 2 → 8 → 10 → 3 → 9 → 12 → 13 → 5 → 4 → 11 -
> 6 → 7 → 14 → 1 | tempo=6m59.840804168s
2026/11/02 14:09:22 n=15 | rotas=87178291200 (teórico=87178291200) | arestas=1307674368000 | custo mínimo=124 | rota=1 → 2 → 8 → 10 → 3 → 9 → 6 → 11 → 4 → 5 →
13 → 12 → 15 → 7 → 14 → 1 | tempo=1h39m41.114958103s
2026/11/02 14:09:22 Benchmark concluído. Arquivos: resultados/execucao_2343673253.log e resultados/resultados_81501847.csv
```

Figura 3: Saída da execução CPU, incluindo as contagens e tempos registrados.

```
2026/10/31 15:17:30 TSP por força bruta (GPU) | origem fixa: cidade 1 | semente: 42
2026/10/31 15:17:30 Resultados serão gravados em resultados
2026/10/31 15:17:31 n=3 | rotas=2 (teórico=2) | arestas=6 | custo mínimo=96 | rota=1 → 2 → 3 → 1 | tempo=161.061415ms
2026/10/31 15:17:31 n=4 | rotas=6 (teórico=6) | arestas=24 | custo mínimo=88 | rota=1 → 2 → 3 → 4 → 1 | tempo=65.13µs
2026/10/31 15:17:31 n=5 | rotas=24 (teórico=24) | arestas=120 | custo mínimo=100 | rota=1 → 2 → 3 → 4 → 5 → 1 | tempo=67.49µs
2026/10/31 15:17:31 n=6 | rotas=120 (teórico=120) | arestas=720 | custo mínimo=85 | rota=1 → 2 → 3 → 4 → 5 → 6 → 1 | tempo=61.299µs
2026/10/31 15:17:31 n=7 | rotas=720 (teórico=720) | arestas=5040 | custo mínimo=112 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 1 | tempo=65.35µs
2026/10/31 15:17:31 n=8 | rotas=5040 (teórico=5040) | arestas=40320 | custo mínimo=118 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 1 | tempo=63.5µs
2026/10/31 15:17:31 n=9 | rotas=40320 (teórico=40320) | arestas=362880 | custo mínimo=165 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 1 | tempo=102.229µs
2026/10/31 15:17:31 n=10 | rotas=362880 (teórico=362880) | arestas=3628800 | custo mínimo=168 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 1 | tempo=568.498µs
2026/10/31 15:17:31 n=11 | rotas=3628800 (teórico=3628800) | arestas=39916800 | custo mínimo=153 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 1 |
tempo=3.807393ms
2026/10/31 15:17:31 n=12 | rotas=39916800 (teórico=39916800) | arestas=479001600 | custo mínimo=166 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 →
1 | tempo=40.446765ms
2026/10/31 15:17:31 n=13 | rotas=479001600 (teórico=479001600) | arestas=6227020800 | custo mínimo=181 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 -
> 13 → 1 | tempo=493.333829ms
2026/10/31 15:17:38 n=14 | rotas=6227020800 (teórico=6227020800) | arestas=87178291200 | custo mínimo=170 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 →
12 → 13 → 14 → 1 | tempo=7.256399898s
2026/10/31 15:19:35 n=15 | rotas=87178291200 (teórico=87178291200) | arestas=1307674368000 | custo mínimo=190 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11
→ 12 → 13 → 14 → 15 → 1 | tempo=1m56.386010315s
2026/10/31 15:46:37 n=16 | rotas=1307674368000 (teórico=1307674368000) | arestas=20922789888000 | custo mínimo=204 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 -
> 11 → 12 → 13 → 14 → 15 → 16 → 1 | tempo=27m2.276604931s
2026/10/31 23:17:52 n=17 | rotas=20922789888000 (teórico=20922789888000) | arestas=355687428096000 | custo mínimo=219 | rota=1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 →
10 → 11 → 12 → 13 → 14 → 15 → 16 → 17 → 1 | tempo=7h31m14.814512025s
```

Figura 4: Saída da execução GPU, incluindo as contagens e tempos registrados.

6 Discussão das questões investigativas

6.1 Redução do espaço de busca

Sem fixar a cidade inicial, seriam enumeradas $n!$ permutações. Como cada circuito pode ser escrito começando por qualquer uma das suas n cidades, essas permutações incluem n rotações do mesmo circuito. Fixar v_1 elimina essa redundância e deixa $(n-1)!$ rotas, reduzindo por um fator n. Como cada rota exige avaliar n arestas, sem a fixação seriam feitas $n \cdot n!$ avaliações de arestas; com a fixação, são $n \cdot (n-1)! = n!$.

O código reflete essa redução: tanto a versão sequencial quanto a GPU mantêm a cidade de início fixa (cidade 1 nos testes principais) e permutam apenas as demais. Para $n = 10$, por exemplo, a CPU e cada passagem da GPU avaliaram $9! = 362.880$ rotas e $10! = 3.628.800$ arestas. Sem fixar a origem, seriam $10! = 3.628.800$ representações de rotas e $10 \cdot 10! = 36.288.000$ avaliações de arestas por passagem, sem obter novos circuitos distintos nem melhorar o custo ótimo. As contagens históricas observadas coincidem com os valores teóricos por passagem para todos os tamanhos testados.

Tabela 3: Comparação teórica do espaço de busca com e sem origem fixa.

n	Rotas fixas	Rotas sem fixar	Arestas fixas	Arestas sem fixar
3	2	6	6	18
6	120	720	720	4.320
10	362.880	3.628.800	3.628.800	36.288.000
15	$8{,}72 \times 10^{10}$	$1{,}31 \times 10^{12}$	$1{,}31 \times 10^{12}$	$1{,}96 \times 10^{13}$
17	$2{,}09 \times 10^{13}$	$3{,}56 \times 10^{14}$	$3{,}56 \times 10^{14}$	$6{,}05 \times 10^{15}$

Os valores da Tabela 3 são calculados pelas fórmulas por passagem; não representam medições sem fixação. A Figura 5 também é teórica. O modo de todas as origens permite verificar empiricamente esses valores: na CPU, contam-se $n!$ rotas e $n \cdot n!$ arestas; na GPU corrigida, são $2n!$ avaliações de rotas e $2n \cdot n!$ arestas nas duas passagens. O apêndice inclui automaticamente os tempos sem fixação quando houver CSVs desse modo, e o gerador confere suas contagens. Até a execução desses testes, não se deve apresentá-los como resultados observados.

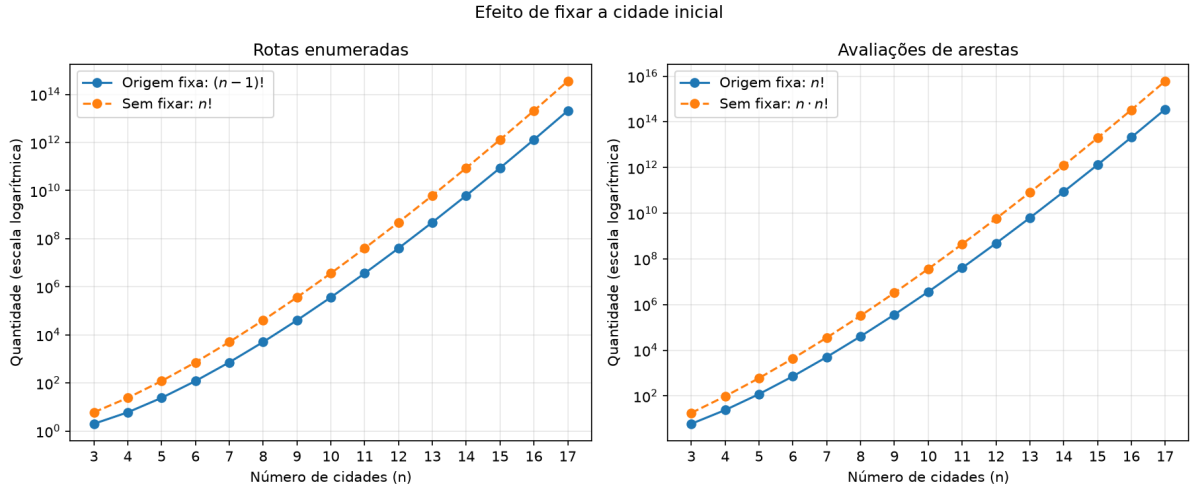


Figura 5: Quantidade teórica de rotas e avaliações de arestas com origem fixa e sem fixá-la.

6.2 Custo de avaliação por rota

A avaliação percorre as n arestas do circuito, inclusive o retorno à origem. Para $n = 10$, foram avaliadas 362.880 rotas e 3.628.800 arestas ($10 \cdot 362.880 = 10!$) na CPU e em cada passagem GPU. Na CPU, a avaliação e montagem da rota custam $\Theta(n)$ por rota, e a busca completa custa $\Theta(n!)$. Na GPU, embora haja n consultas de arestas por rota, a reconstrução da permutação também percorre cidades candidatas em laços aninhados, podendo custar $\Theta(n^2)$ por rota. Logo, a contagem de arestas isoladamente não descreve todo o trabalho nem o tempo paralelo da GPU.

6.3 Limiar prático da intratabilidade

Na CPU, $n = 13$ levou em mediana aproximadamente 31,84 s; o primeiro tempo acima de um minuto ocorreu em $n = 14$, com cerca de 7 min 13 s. Para $n = 15$, a única amostra disponível levou aproximadamente 1 h 39 min 41 s. Na GPU, $n = 14$ levou cerca de 6,07

s, $n = 15$ cerca de 1 min 45 s, $n = 16$ cerca de 26 min 33 s e $n = 17$ cerca de 7 h 31 min. Os tempos para os maiores tamanhos são observações de uma amostra e não devem ser tratados como estimativas estáveis; o tempo GPU inclui inicialização e transferências. Os registros das execuções estão nas Figuras 3 e 4.

O Merge Sort é um exemplo de algoritmo com complexidade $\Theta(n \log n)$, enquanto a busca exaustiva sequencial aqui estudada tem crescimento fatorial. A recorrência $T(n) = 2T(n-1) + c$ discutida na seção 4.2 do material é um exemplo distinto de crescimento $\Theta(2^n)$; ela não é a recorrência usada para deduzir o custo deste TSP. A comparação evidencia que fatores polinomiais, exponenciais e fatoriais produzem comportamentos muito diferentes conforme a entrada aumenta.

6.4 Classes P e NP e heurísticas

A classe P reúne problemas de decisão solucionáveis em tempo polinomial; NP inclui problemas de decisão cujas soluções propostas podem ser verificadas em tempo polinomial. A versão de decisão do TSP é NP-completa, enquanto sua versão de otimização é NP-difícil. Não se conhece um algoritmo polinomial exato geral para o problema, mas isso não prova que tal algoritmo não exista: a questão $P = NP$ permanece em aberto.

Uma heurística gulosa pode escolher, em cada passo, uma cidade que pareça vantajosa localmente. Essa escolha não garante a melhor combinação global e, portanto, não assegura o custo mínimo para todas as instâncias. Heurísticas e metaheurísticas podem produzir soluções úteis em tempo menor, mas sua qualidade e suas garantias dependem do método empregado.

7 Conclusão

O estudo confirmou a enumeração de $(n-1)!$ rotas e a avaliação de $n!$ arestas por passagem. A GPU executa duas passagens, embora os contadores dos CSVs históricos registrem apenas a primeira; a versão corrigida do código registra ambas. A implementação CPU concluiu até $n = 15$ e a GPU até $n = 17$, mas ambas apresentaram crescimento acentuado do tempo, ilustrando a intratabilidade prática da busca exaustiva. A GPU concluiu instâncias maiores no equipamento testado, mas os ambientes distintos, o número desigual de amostras e os custos de inicialização e transferência impedem uma comparação rigorosa de desempenho. As medianas agregadas são mais informativas que uma amostra isolada quando há repetições; onde só há uma execução, a limitação foi mantida explícita. Uma avaliação futura deve repetir cada tamanho várias vezes e, para comparar CPU e GPU, usar o mesmo computador e as mesmas condições.

Referências

[1] UNIVERSIDADE DE VILA VELHA. *Projeto e Análise de Algoritmos: Unidade 4.* Material da disciplina, seções 4.1 e 4.2.

[2] *Proposta de Atividade Investigativa: Explorando a Intratabilidade e a Força Bruta no Problema do Caixeiro Viajante (TSP).* Documento fornecido para a atividade.

[3] CABRITTO CORPS. *tsp-go: implementação do Problema do Caixeiro Viajante*. Repositório de código-fonte. Disponível em: `https://github.com/Cabritto-Corps/tsp-go`.

A Tempos individuais e dispersão

A.1 Medições individuais: CPU, origem fixa

As letras identificam os arquivos de origem; tempos em milissegundos. Mínimo, mediana e máximo estão arredondados a três algarismos significativos.

- A: `resultados/resultados.csv`
- B: `resultados/resultados_3879801335.csv`
- C: `resultados/resultados_477070715.csv`
- D: `resultados/resultados_81501847.csv`
- E: `resultados/resultados_862872379.csv`
- F: `resultados/resultados_972862127.csv`

n	Amostras (arquivo: tempo)	Mín.	Mediana	Máx.
3	A: 0,000630; B: 0,006473; C: 0,005932; D: 0,000701; E: 0,000902; F: 0,000942	0.00063	0.000922	0.00647
4	A: 0,003730; B: 0,001112; C: 0,000481; D: 0,000471; E: 0,000582; F: 0,000511	0.000471	0.000547	0.00373
5	A: 0,001180; B: 0,002365; C: 0,001683; D: 0,001052; E: 0,001042; F: 0,001183	0.00104	0.00118	0.00236
6	A: 0,005430; B: 0,007114; C: 0,007415; D: 0,004709; E: 0,005099; F: 0,005109	0.00471	0.00527	0.00741
7	A: 0,048309; B: 0,057766; C: 0,035712; D: 0,026098; E: 0,027983; F: 0,027643	0.0261	0.0318	0.0578
8	A: 0,384213; B: 0,793352; C: 0,263428; D: 0,422930; E: 0,197861; F: 0,203407	0.198	0.324	0.793
9	A: 3,575375; B: 3,311486; C: 4,599842; D: 2,469023; E: 2,937508; F: 2,379585	2.38	3.12	4.6
10	A: 21,730290; B: 23,454852; C: 24,148688; D: 20,703818; E: 20,937891; F: 22,507273	20.7	22.1	24.1

n	Amostras (arquivo: tempo)	Mín.	Mediana	Máx.
11	B: 241,149517; C: 234,589688; D: 220,352922; E: 241,903932; F: 213,259329	213	235	242
12	B: 2816,919732; C: 2675,783333; D: 2467,607679; E: 2690,764910; F: 2529,056042	2.47e+03	2.68e+03	2.82e+03
13	B: 32885,400771; D: 30599,227790; E: 32980,531361; F: 30787,241502	3.06e+04	3.18e+04	3.3e+04
14	B: 450664,889607; D: 419840,804168; E: 444462,081083; F: 421648,064272	4.2e+05	4.33e+05	4.51e+05
15	D: 5981114,958103	5.98e+06	5.98e+06	5.98e+06

Contagens efetivamente registradas nos CSVs (rotas e arestas):

n	Avaliações de rotas	Avaliações de arestas
3	2	6
4	6	24
5	24	120
6	120	720
7	720	5040
8	5040	40320
9	40320	362880
10	362880	3628800
11	3628800	39916800
12	39916800	479001600
13	479001600	6227020800
14	6227020800	87178291200
15	87178291200	1307674368000

A.2 Medições individuais: GPU, origem fixa

As letras identificam os arquivos de origem; tempos em milissegundos. Mínimo, mediana e máximo estão arredondados a três algarismos significativos.

- A: `resultados/results_gpu.csv`
- B: `resultados/results_gpu_2372117759.csv`

n	Amostras (arquivo: tempo)	Mín.	Mediana	Máx.
3	A: 161,061415; B: 142,841025	143	152	161
4	A: 0,065130; B: 0,357791	0.0651	0.211	0.358
5	A: 0,067490; B: 1,408151	0.0675	0.738	1.41
6	A: 0,061299; B: 0,635886	0.0613	0.349	0.636

n	Amostras (arquivo: tempo)	Mín.	Mediana	Máx.
7	A: 0,065350; B: 0,302767	0.0654	0.184	0.303
8	A: 0,063500; B: 0,503650	0.0635	0.284	0.504
9	A: 0,102229; B: 0,355900	0.102	0.229	0.356
10	A: 0,568498; B: 0,575293	0.568	0.572	0.575
11	A: 3,807393; B: 2,853125	2.85	3.33	3.81
12	A: 40,446765; B: 24,476475	24.5	32.5	40.4
13	A: 493,333829; B: 311,104639	311	402	493
14	A: 7256,399898; B: 4890,258530	4.89e+03	6.07e+03	7.26e+03
15	A: 116386,010315; B: 93118,212972	9.31e+04	1.05e+05	1.16e+05
16	A: 1622276,604931; B: 1563253,068073	1.56e+06	1.59e+06	1.62e+06
17	A: 27074814,512025	2.71e+07	2.71e+07	2.71e+07

Contagens efetivamente registradas nos CSVs (rotas e arestas):

n	Avaliações de rotas	Avaliações de arestas
3	2	6
4	6	24
5	24	120
6	120	720
7	720	5040
8	5040	40320
9	40320	362880
10	362880	3628800
11	3628800	39916800
12	39916800	479001600
13	479001600	6227020800
14	6227020800	87178291200
15	87178291200	1307674368000
16	1307674368000	20922789888000
17	20922789888000	355687428096000